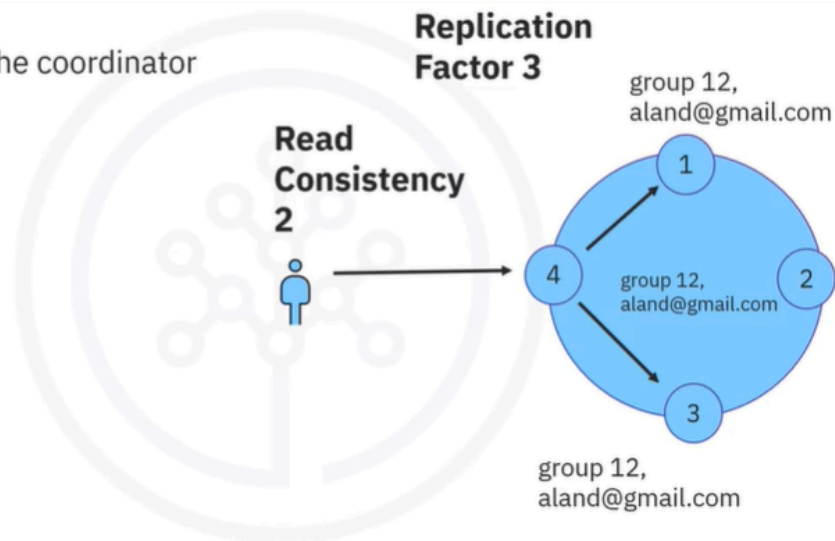


CRUD Operations - Part 2

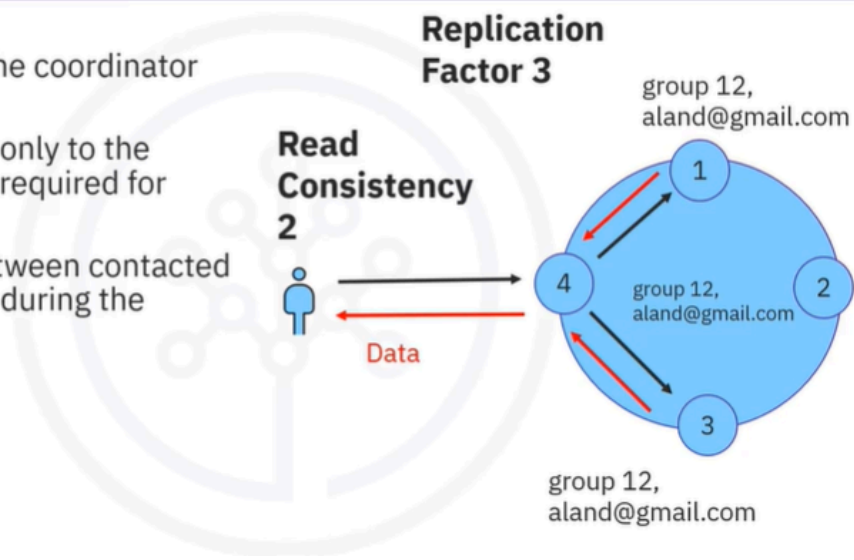
Read operations in Cassandra

- Receiving node is the coordinator for the operation



Read operations in Cassandra

- Receiving node is the coordinator for the operation
- Reads are directed only to the number of replicas required for consistency
- Inconsistencies between contacted nodes are repaired during the Read process



- Now that we have learned about how to insert data, let's see how to read the data. When a read operation is directed to a node in the cluster, that node becomes the coordinator of the read operation and is responsible for its completion. In our example, node 4 is the coordinator for this read.
- Reads are sent only to the number of replicas specified by the consistency setting. For example, a consistency of two means that only two nodes of the three replicas will be contacted. The coordinator will reconcile the responses received from the contacted nodes.

READ/SELECT rules in Cassandra

- Start your query using the Partition Key
 - Limit Reads to specific nodes containing your data
 - Follow the order of your Primary Key columns
- Primary Key(PartitionKey, ClusteringKey1, ClusteringKey2)

```

SELECT * FROM table WHERE PartitionKey = ;
SELECT * FROM table WHERE PartitionKey IN (,); //list of values
SELECT * FROM table WHERE PartitionKey = AND ClusteringKey1 = ; OK
SELECT * FROM table WHERE PartitionKey = AND ClusteringKey1
= AND ClusteringKey2 = ;
SELECT * FROM groups; // not okay performance wise
SELECT * FROM groups WHERE clustering key1 = ; // will not work
SELECT * FROM groups WHERE regular_column = ; // will not work

```

**NOT OK
ERROR***

* Unless ALLOW FILTERING

- If there are any inconsistencies, they will be resolved based on the operations time stamps, then the result will be sent back. Syntax-wise, in Cassandra, just like in many databases, reads are performed using the select operation. There are a few rules when it comes to select operations in Cassandra. Always start your query with the partition key to limit your read to only the replicas for your partition.
- Follow your primary key fields order in your query to get the best performance. For example, if you have a primary key formed of one partition key and two clustering keys, filtering a specific value of the partition key is

okay. Note that partition key supports only equal and in operations. Filtering a list of possible values for the partition key is also okay.

- Filtering the partition key and the first clustering key is okay. Filtering the partition key and the first and second clustering keys will just read one record, which is okay. However, selecting all data from your table is not okay in production systems because this will send the request to all nodes in the cluster. Imagine having a 1,000 node cluster, would you be happy with the performance of your query? Your query will work, but the performance will be really bad, even if no timeout occurs. Filtering a regular column is not okay and will not work. Finally, filtering the clustering keys without the partition key is not okay and will not work.

Secondary indexes

```
USE intro_cassandra;  
SELECT * FROM groups WHERE groupid=12;  
SELECT * FROM groups WHERE groupid=12 and username='aland@gmail.com';
```

```
SELECT * FROM groups WHERE age = 12; //will not work  
CREATE INDEX ON groups(age);  
SELECT * FROM groups WHERE age = 12; //will work
```

```
SELECT * FROM groups WHERE groupid=12 AND age = 12;
```

Groups
PRIMARY KEY
(groupid,username)

A best practice is
to use indexes
inside a partition

Always start your queries with the partition key

- Let's look at some examples based on our groups table where the primary key is composed of group ID and username columns. Filtering the group ID or group ID and username will work and yield a very good performance.
- Filtering the age column, which is a regular one, will not work, Cassandra will return an error. What you could do in this case is either remodel your table or create an index on column H, and then the select query will work.

- But while the previous query will work, optimal performance will be obtained by filtering the age column together with a specific partition key group ID. When you do this, you limit your query to only the nodes that host that particular partition key. This brings us again to this very important Cassandra rule: always start your queries with the partition key.

DELETE

groupid	username	group_name	age
45	Moriad@yahoo.com	grilling	Null
20	bella@yahoo.com	coffee	55

- Record
- Cell
- Range
- Partition

```
DELETE FROM groups WHERE groupid=12 AND
username='Elaine@yahoo.com';
DELETE age FROM groups WHERE groupid=12
AND username='aland@gmail.com';
SELECT * FROM groups;
DELETE FROM groups where groupid=12;
```

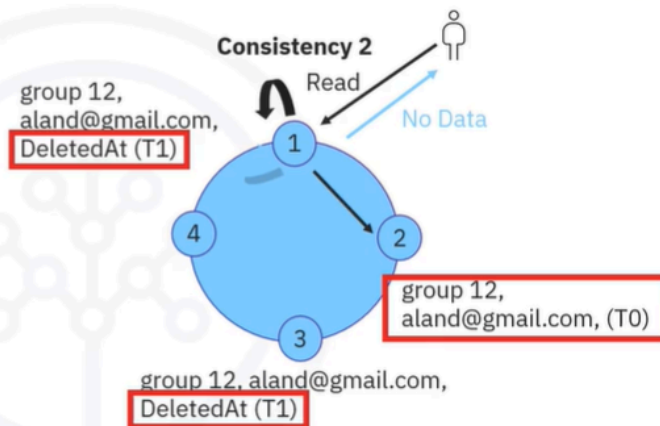
- We're starting off with some data to demonstrate some deletes. In Cassandra, you can delete an entry identified by the full primary key and a cell in an entry identified by the full primary key. If we perform a select from our table, you can see that we do not have the user called Elaine anymore and the age cell has been deleted for the user called Allan. Deletes can also be done at the partition level. You can delete a continuous range in a partition. If your partition is sensor ID and your data is clustered by the time of the recording, you can, for example, delete all entries on a sensor between 1:00 PM and 3:00 PM, or you can delete a whole partition which is identified by a partition key.

DELETE

- Deleting data in distributed systems is trickier than in relational databases
 - Especially in peer-to-peer databases like Cassandra
 - Reads and writes can be directed to any of partition's replicas, so there's no primary node for a writes or reads
 - Cassandra marks all deleted items with a "tombstone" containing the time of the delete operation
-
- We've seen the syntax of deleting data, but in Cassandra, you should use delete operation sparingly because their usage greatly affects system performance. Deleting distributed and replicated data from a system such as Apache Cassandra is far trickier than in a relational database, especially if we remember that Cassandra is a peer-to-peer architecture, where reads and writes can be directed to any node in the cluster, so there's no primary node for writes or reads.
 - In such a system, to record the fact that a delete happened, a special value called a tombstone needs to be written as an indicator that previous values are to be considered deleted. From the moment an item has been marked with a tombstone, its data is not visible any longer to queries. However, the actual data will still reside on disk for a period of time.

Tombstones

- A special value to indicate data has been deleted + time of delete
- Tombstones prevent deleted data from being returned during reads
- Tombstones are deleted at 'gc_grace_seconds' during compaction process



- What are tombstones? In Cassandra, a delete is just a write operation that also has a special value appended to indicate that the data has been deleted and the time of the delete. This value is called a tombstone. Here, as you can see in the diagram, our delete was acknowledged by nodes 1 and 3, and now we have a tombstone indicating that our specific primary key data was deleted at T1. Node 2 data is the initial one, the insert done at T0 since the delete operation did not succeed on node 2.
- Now, when a read operation arrives, the data will be returned by nodes 1 and 2. Node 1 will return no data with time T1 and node 2 will return requested data with time T0. T1 is more recent than T0. So the coordinator node, node 1, will decide that the newest data is the one from node 1 and return no data as part of the read. One other thing to remember is that tombstones are deleted only when a configurable period has elapsed. This is called `gc_grace_seconds`, which is set to 10 days by default and is set at the table level.

Recap

In this video, you learned that:

- Cluster-level Reads are only sent to replicas according to consistency setting
- Reads should follow the Primary Key columns order for best performance
- Delete operations can be done at record, cell, range, and partition levels

